

Министерство образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

Отчёт

по лабораторной работе №3

по курсу «Л и О А в ИЗ»

на тему «Динамические списки»

Выполнили студенты группы 24ВВВ4:

Орлов Р.А.

Симаков А.С.

Приняли к.т.н., доцент:

к. т. н., доцент Юрова О.В.

к. э. н., доцент Акифьев И.В

Пенза 2025

Цель: освоить практические навыки работы с динамическими структурами данных на языке С.

Общие сведения.

Список представляет собой последовательность элементов определенного типа. Простейший тип списка – линейный, когда для каждого из элементов, кроме последнего, имеется следующий, и для каждого, кроме первого, имеется предыдущий элемент.

Возможна реализация списков посредством массивов или динамическая реализация.

Динамические списки относятся к динамическим структурам и используются, когда размер данных заранее неизвестен. Созданием динамических данных должна заниматься сама программа во время своего исполнения, этим достигается эффективно распределение памяти, но снижается эффективность доступа к элементам.

Динамические структуры данных отличаются от статических двумя основными свойствами:

- 1) в них нельзя обеспечить хранение в заголовке всей информации о структуре, поэтому каждый элемент должен содержать информацию, логически связывающую его с другими элементами структуры;
- 2) для них зачастую не удобно использовать единый массив смежных элементов памяти, поэтому необходимо предусматривать ту или иную схему динамического управления памятью.

Для обращения к динамическим данным применяют указатели.

Набор операций над списком будет включать добавление и удаление элементов, поиск элементов списка.

Различают односвязные, двусвязные и циклические списки.

В простейшем случае каждый элемент содержит всего одну ссылку на следующий элемент, такой список называется односвязным.

В простейшем случае для создания элемента списка используется структура, в которой объединяются полезная информация и ссылка на следующий элемент списка

Задание

1) Реализовать приоритетную очередь, путём добавления элемента в список в соответствии с приоритетом объекта (т.е. объект с большим приоритетом становится перед объектом с меньшим приоритетом).

2) *На основе приведенного кода реализуйте структуру данных *Очередь*.

3) *На основе приведенного кода реализуйте структуру данных *Стек*.

Листинг

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <locale.h>
#include <stdlib.h>
#include <string.h>

struct node
{
    char inf[256];
    int priority;
    struct node* next;
};

struct node* head = NULL;
struct node* last = NULL;

// Функция создания нового элемента
```

```

struct node* get_struct(void)
{
    struct node* p = NULL;
    char s[256];
    int priority;

    if ((p = (struct node*)malloc(sizeof(struct node))) == NULL)
    {
        printf("Ошибка при распределении памяти\n");
        exit(1);
    }

    printf("Введите название объекта: ");
    scanf("%255s", s); // Безопасный ввод
    getchar(); // Очистка буфера

    printf("Введите приоритет объекта (целое число): ");
    scanf("%d", &priority);
    getchar(); // Очистка буфера

    if (strlen(s) == 0)
    {
        printf("Запись не была произведена\n");
        free(p);
        return NULL;
    }

    strcpy(p->inf, s);
    p->priority = priority;
    p->next = NULL;

```

```
    return p;
}
```

// Функция добавления элемента в приоритетную очередь

```
void priority_enqueue(void)
```

```
{
    struct node* p = NULL;
    struct node* current = NULL;
    struct node* prev = NULL;
```

```
    p = get_struct();
```

```
    if (p == NULL)
```

```
    {
        return;
    }
```

// Если список пуст или новый элемент имеет высший приоритет

```
if (head == NULL || p->priority > head->priority)
```

```
{
    p->next = head;
    head = p;
    if (p->next == NULL) {
        last = p;
    }
    return;
}
```

// Поиск места для вставки

```
current = head;
```

```

while (current != NULL && current->priority >= p->priority)
{
    prev = current;
    current = current->next;
}

// Вставка элемента
prev->next = p;
p->next = current;

// Если элемент стал последним
if (current == NULL)
{
    last = p;
}
}

// Функция добавления элемента в обычную очередь (в конец)
void enqueue(void)
{
    struct node* p = NULL;
    p = get_struct();
    if (p == NULL)
    {
        return;
    }

    p->priority = 0; // Для обычной очереди приоритет не важен

    if (head == NULL)

```

```
{
    head = p;
    last = p;
}
else
{
    last->next = p;
    last = p;
}
}
```

// Функция просмотра списка

void review(void)

```
{
    struct node* struc = head;
    if (head == NULL)
    {
        printf("Список пуст\n");
        return;
    }
}
```

printf("Содержимое списка:\n");

while (struc)

```
{
    printf("Имя: %s, Приоритет: %d\n", struc->inf, struc->priority);
    struc = struc->next;
}
}
```

// Функция извлечения элемента из начала списка (для обеих очередей)

```

struct node* dequeue(void)
{
    struct node* temp = head;
    if (head == NULL)
    {
        printf("Очередь пуста\n");
        return NULL;
    }

    head = head->next;
    if (head == NULL)
    {
        last = NULL;
    }
    temp->next = NULL;
    return temp;
}

// Функция поиска элемента по имени
struct node* find(char* name)
{
    struct node* struc = head;
    if (head == NULL)
    {
        printf("Список пуст\n");
        return NULL;
    }

    while (struc)
    {

```



```

        if (strcmp(name, struc->inf) == 0)
        {
            return struc;
        }
        struc = struc->next;
    }

    printf("Элемент не найден\n");
    return NULL;
}

// Главная функция
int main()
{
    setlocale(LC_ALL, "rus");
    int choice;
    char name[256];
    struct node* temp;

    while (1)
    {
        printf("\n1. Добавить элемент в приоритетную очередь\n");
        printf("2. Добавить элемент в обычную очередь\n");
        printf("3. Просмотреть очередь\n");
        printf("4. Найти элемент\n");
        printf("5. Извлечь элемент (из начала очереди)\n");
        printf("6. Выход\n");
        printf("Выберите действие: ");
        scanf("%d", &choice);
        getchar(); // Очистка буфера
    }
}

```

```

switch (choice)
{
case 1:
    priority_enqueue();
    break;
case 2:
    enqueue();
    break;
case 3:
    review();
    break;
case 4:
    printf("Введите имя для поиска: ");
    scanf("%255s", name);
    temp = find(name);
    if (temp != NULL)
    {
        printf("Найден: %s с приоритетом %d\n", temp->inf, temp-
>priority);
    }
    break;
case 5:
    temp = dequeue();
    if (temp != NULL)
    {
        printf("Извлечен элемент: %s (приоритет: %d)\n", temp->inf,
temp->priority);
        free(temp);
    }
}

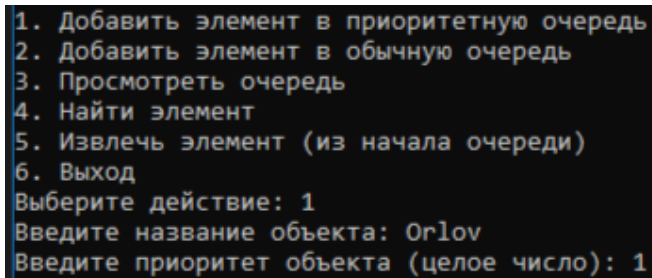
```

```

        break;
    case 6:
        // Очистка памяти перед выходом
        while (head != NULL)
        {
            temp = head;
            head = head->next;
            free(temp);
        }
        exit(0);
    default:
        printf("Неверный выбор\n");
    }
}
return 0;
}

```

Результат работы программы



```

1. Добавить элемент в приоритетную очередь
2. Добавить элемент в обычную очередь
3. Просмотреть очередь
4. Найти элемент
5. Извлечь элемент (из начала очереди)
6. Выход
Выберите действие: 1
Введите название объекта: Orlov
Введите приоритет объекта (целое число): 1

```

Рисунок 1 – Начальное меню

```
Введите приоритет объекта (целое число): 1
1. Добавить элемент в приоритетную очередь
2. Добавить элемент в обычную очередь
3. Просмотреть очередь
4. Найти элемент
5. Извлечь элемент (из начала очереди)
6. Выход
Выберите действие: 1
Введите название объекта: Orlov
Введите приоритет объекта (целое число): 2
1. Добавить элемент в приоритетную очередь
2. Добавить элемент в обычную очередь
3. Просмотреть очередь
4. Найти элемент
5. Извлечь элемент (из начала очереди)
6. Выход
Выберите действие: 5
Извлечен элемент: Orlov (приоритет: 2)
1. Добавить элемент в приоритетную очередь
2. Добавить элемент в обычную очередь
3. Просмотреть очередь
4. Найти элемент
5. Извлечь элемент (из начала очереди)
6. Выход
Выберите действие: 6
```

Рисунок 2 – Присваивание приоритета и показ элемента с наивысшим приоритетом

Вывод: в ходе выполнения данной лабораторной работы освоили практические навыки работы с динамическими структурами данных на языке С. Была реализована приоритетная очередь на основе односвязного списка.